

Exercise 1.2: Using Objects

Lab Objectives

- Start to understand how to create objects and use their methods
- Look at the Java classes
 - String - for storing and working with strings
 - Scanner - for reading input from the keyboard
- Become accustomed with the Java API

Classes and Objects

Java is an object-oriented programming language and whilst you will be learning about the theoretical principles behind this in your lectures, it is worth covering the basics prior to your work for this lab exercise.

Every variable in Java represents either a primitive type or a reference type. The value associated with a reference type is a reference to an object. An object is an instance of a class and a class is therefore a template, which defines an object.

There are literally thousands of classes within the Java standard library, ready for developers to make use of. When a developer wants to use a particular class, they generally need to create an object instance of that class by calling the class's constructor. The constructor will allocate memory to store an object of that class type, initialise the internal state of that object and then return an object reference to be stored as the value of the associated variable. It is possible and often necessary to make several different object instances of the same class; each will have its own memory allocation and internal state.

The class template defines a public interface, which defines the behaviour of objects associated with that class. A public interface is a set of methods, which are usually used to access or modify the internal state of an object. A method is in essence a function, i.e. the developer calls a method and may pass parameters to it or expect a return value. Calling or as is commonly said, invoking a method causes a message to be sent to the receiver object, for which the method belongs.

The Java Application Programming Interface (API) provides documentation for every class within the standard library. It tells developers how they can use classes, by detailing how to use the public interface. For example how methods for a given class will behave answering questions such as: What does it actually do? Is there a return value? What parameters does it accept?

Please continue to the next page...

Using the Java String type

The String type in Java is a special type. Although there is a String class and therefore it is possible to create a new String object by calling the class's constructor, there is another more convenient way, which does not require a call to the constructor and instead creates what is called a String Literal. This means that strings can be created in a similar way to primitive types.

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/java/data/strings.html>

Note: you will find further reading resources at the end of this week's slides, e.g. associated Oracle tutorials, and will also find useful syntax examples within the slides too.

Question 1 Reopen your Eclipse workspace called "CTEC2710_OO" and then download the existing Java Project required for this lab by extracting it from the "UsingObjects.zip" file, and importing the project into Eclipse. As a reminder you need to go to **File > Import... > General > Existing Projects into Workspace > Browse** and locate the UsingObjects project folder, then press **Finish**, ensuring you select "Copy projects into workspace".

Within the **strings** package, study the source code and comments in the "**StringDemo**" program, and give it a run to see the current output.

The program so far has been commented to assist with your understanding. Some of the String methods have been used, so to look at them in detail go to the Java API: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/String.html>

You may read the class overview, then look at the **Method Summary**. Here you will see a list of all the methods belonging to the String class, some of which you have just used. The Method Summary provides the fundamental details, notably the method header, which lists the return type and any parameters the method accepts.

Method Summary	
Methods	
Modifier and Type	Method and Description
char	charAt(int index) Returns the char value at the specified index.
int	codePointAt(int index) Returns the character (Unicode code point) at the specified index.
int	codePointBefore(int index) Returns the character (Unicode code point) before the specified index.
int	codePointCount(int beginIndex, int endIndex) Returns the number of Unicode code points in the specified text range of this String.
int	compareTo(String anotherString) Compares two strings lexicographically.
int	compareToIgnoreCase(String str) Compares two strings lexicographically, ignoring case differences.
String	concat(String str) Concatenates the specified string to the end of this string.
boolean	contains(CharSequence s) Returns true if and only if this string contains the specified sequence of char values.

You can click on an individual method name, e.g. **charAt**, and you will then be provided with more detail about the behaviour of the method and therefore how it should be used.

charAt

```
public char charAt(int index)
```

Returns the char value at the specified index. An index ranges from 0 to `length() - 1`. The first char value of the sequence is at index 0, the next at index 1, and so on, as for array indexing.

If the char value specified by the index is a surrogate, the surrogate value is returned.

Specified by:

`charAt` in interface `CharSequence`

Parameters:

`index` - the index of the char value.

Returns:

the char value at the specified index of this string. The first char value is at index 0.

It can be seen that the `charAt` method accepts a parameter value of type `int`, which should represent an index in the `String` and then the corresponding character value stored at that index will be returned as a `char` type (e.g. 0 represents the first character). There is also information about what will happen should an invalid value be passed to the method, although you are not expected to understand exceptions at the point in time.

Summary - Using the Java API:

- The API has given us all the information we need to understand how the **charAt(...)** method works, without which we would have had to guess.
- For such a simple method it may have been obvious, but for more complex methods it may not be and simply making an informed guess at how a method works, instead of checking how it actually works, is strictly not advised as this can obviously lead to unexpected results.

Question 2

Using your **StringDemo** class from the previous question:

- Further your understanding of the `String` class by looking at the [API documentation](#) for each of the remaining methods used in `StringDemo`, i.e. **length()** and **endsWith(...)**. Find out how the methods work, any parameters they accept and their return type.
- Experiment with some other methods that belong to the `String` class, you should try to use each of the methods listed below, using the Java API to check their specification.

```
public String toUpperCase()  
public String replace(char oldChar, char newChar)  
public String substring(int beginIndex, int endIndex)  
public int indexOf(String str)
```

Question 3

In the `strings` package, create a new class called **"Initials"** and then:

- Write a program to output the initials of a name stored within a `String` e.g. if name is "David Beckham", then the initials are "DB". **Hint:** You may wish to make use of the **charAt()** and **indexOf()** methods.
- Extend the program to construct and output an email address for a given name. The email address should be in lower case, and take the form *initials@email.dmu.ac.uk*. E.g. If the

person name is David Beckham then the email is *db@email.dmu.ac.uk*. **Hint:** Think of how you may change the initials to lower case using the methods available to you.

Question 4 It was previously stated that every variable in Java represents either a primitive type or a reference type. There are three main reference types; class types, interface types (which we will explore later in the module) and array types. Regardless of what it stores (primitive or not) an array represents a container object and therefore it needs to be instantiated in a similar way that a class does (by using the 'new' keyword).

Within the strings package of your current project create a new class called "**StringArrayDemo**", then within the main method do the following:

- a) Declare an array to store 6 String objects (`String[] fruit = new String[6];`). Then assign the name of a fruit to each element by assigning a string literal (`fruit[0] = "grape";` etc).
- b) Using a loop, print out each element in the array in upper case.

Notes - Shorthand array initialization:

- There is a shorthand way of initializing an array if you know all required values, e.g.

```
int[] numbers = {1, 5, 7, 3};  
String[] fruit = {"apple", "orange", "pear"};
```
- The size of the array will be determined by the number of elements provided and the **new** keyword is not required as it will be automatically implied by the compiler.

Question 5 Study the code and comments in the **StringLiteralOverview** class, then run the program to ascertain if the results are as you expect.

Notes - Creating and comparing String objects:

- All objects are created by using the new keyword to call a constructor - this will allocate memory for a new object instance. The String class has an exception to this.
- A simpler way to create a String and one that is **recommended** is to use String literals - this creates a String object in a central place called the String Literal Pool, such that if another String literal with the same value is referred to, it uses the same object reference, instead of creating a new object.
- The String Literal Pool has a list of references to String objects, that are shared whenever two literals exist with the identical String value - this naturally increases efficiency at runtime.
- Primitive types in Java such as int, can safely be compared using the == operator. However when objects (i.e. reference types) are compared using the == operator, their value is not compared but instead their references (i.e. memory addresses).
- To compare the contents of an object (i.e. its internal state) the **.equals(...)** method should be used. You will learn more about using this method later in the module.

Question 6 In the strings package, create a new class called "**ImmutableDemo**":

- a) Predict the output of the program fragment below. Was the output what you expected?

```
String str = "HELLO";  
System.out.println(str);  
str.toLowerCase();  
System.out.println(str);
```

- b) Change the 3rd line to `str = str.toLowerCase();` and then run the program again. The output should now be what you expect - do you understand what is happening? Read the first two short paragraphs of the Java API for [java.lang.String](https://docs.oracle.com/javase/7/docs/api/java/lang/String.html).

Notes - Immutable:

- The word *immutable* has the opposite meaning of *mutable*, where mutable suggests something can mutate and thus change in state. An immutable object is one where its internal state cannot be changed after creation - in this sense it acts like a constant
- The String class was designed to be immutable so that object references can be safely shared - this assists the idea of runtime optimization using String Literals as discussed earlier.
- When a String method such as **toLowerCase()** is invoked on a String object instance, it does not change the internal state (i.e. value) of that String, but instead the method returns a reference to a new String with the changes applied.
- By assigning this new object reference returned to the original String variable, the variable's value will be overwritten storing the new reference and thus containing the updated character sequence. Conversely if the return value is not used then the original String variable will appear unchanged.

Please turn over...

Reading keyboard input using the Scanner class

So far you have seen how to use a basic output stream, providing output to the console within Eclipse via **System.out**'s **print** and **println** methods, but you have not seen how to acquire the standard input stream. You may remember one of the ways to do this in Scala was to use the read methods, e.g. `readInt`. In Java there is a class called `Scanner`, which has a set of methods to aid us with reading input streams.

Notes - Scanner overview:

- class `Scanner` in the `java.util` package allows us to get values from an input stream.
- `System.in` is the default input stream, i.e. the keyboard.
- The parameter passed to the `Scanner` constructor is the input stream to read from, e.g.

```
Scanner sc = new Scanner(System.in); //read from keyboard
```
- In later exercises you will see how the `Scanner` can also be passed different types of input stream, e.g. passing a filename (e.g. `new File("data.txt")`) as a parameter.
- A `Scanner` has a **`next()`** method that fetches the next token from the input stream. Unless otherwise specified, tokens are strings that are separated by whitespace (spaces, tabs, newline).
- Other common methods are **`nextInt()`**, **`nextDouble()`** that read the next sequence of characters as a number. Errors occur if the characters are not digits.
- E.g.

```
//create scanner to read from the keyboard
Scanner sc = new Scanner(System.in);
String s = sc.next();           //get the next token as a String
int x = sc.nextInt();           //get the next token as an int
double d = sc.nextDouble();     //get the next token as a double
boolean b = sc.nextBoolean();   //get the next token as a boolean
String line = sc.nextLine();    //read to the end of line
```

Associated Reading:

Java Oracle Tutorials,

<https://docs.oracle.com/javase/tutorial/essential/io/scanning.html>

Question 7

Find the Java API documentation for the `Scanner` class so that you can look at its constructors and methods. To do this go to the API home page for Java 11: <https://docs.oracle.com/en/java/javase/17/docs/api/>. You should familiarise yourself with this page as you can then use it to find classes we use throughout the module or those you wish to explore further yourself.

You will initially see a list of modules – the majority of the resources we will use are in the [java.base](#) module. Each module shows a list of packages, with a general description of the classes located within each of them. The **`Scanner`** class is in the **`java.util`** package, so if you click on `java.util`, you should then see classes solely within this package and be able to select `Scanner`. Alternatively, you can use a search bar (top-right).

Notes - Packages in Java:

- There are thousands of predefined classes within the current Java Standard Edition library, therefore they have sensibly been placed into logical packages where classes with common features or that meet a similar general purpose can be grouped together. Packages themselves are grouped into logical modules, providing additional modularity.
- By default the **java.lang** package is included in all your Java programs, which is why you did not need to **import** a package to use the String class - it exists in java.lang.
- The Scanner class exists in the package java.util and therefore to use it you need to import **java.util.Scanner**, or indeed java.util.* would import all classes within that package if you were using several in a single program.

Question 8 In the **scanner** package, study and then extend the **ScannerDemo** program so that it retrieves some more information such as last name, gender, etc.

Question 9 Open the **ScannerSurprise** class then:

- a) Study the code and run the program - can you understand why you are not prompted to enter the second sentence?
- b) Based upon the comments in the program, fix the issue by adding another call to the **nextLine()** method between reading the number and second sentence.
- c) Refer back to the Java API for the Scanner class, specifically looking at the [nextLine\(\)](#) method. This should clearly state the expected method behaviour, therefore the method does what it is supposed to do and it is up to you as the developer and user of someone else's class to make sure you carefully read the documentation it provides.

Question 10 (*optional challenge*) Write a simple guessing game. Firstly, create a new Java class with an appropriate name, then inside the main method assign a number in the range 1 to 20 to a variable called *target*. Then prompt the user to guess the number. Tell the user whether the guess is higher or lower than the *target*. If the guess is correct then the game is over, congratulate the player and make some comment depending on how many guesses the player took. E.g. "12 is the correct answer. You took 5 guesses. Star rating: Below Average".

Question 11 (*optional challenge*) In a new Java class with an appropriate name, write a program that asks the user to type in a sentence followed by the Enter key, then work out and output:

- a) the number of words in the sentence;
- b) the number of vowels;
- c) the percentage of letters that are vowels.